PYTHON FOR BEGINNERS · SESSION 13

Modules and Packages

import, math, random, datetime, your own modules and pip install.

Recap

Ask the class:

- What does `*args` collect, and what type is it?
- What does `lambda n: n * n` do?
- What is the difference between `map` and `filter`?

What is a module?

A **module** is a Python file full of ready-made code: functions, values and more. Instead of writing everything yourself, you **import** a module and use what is inside.

- Python comes with hundreds of modules. This is called the **standard library**: "batteries included".
- Other people share **packages** (groups of modules) that you can install.
- You can also write **your own** modules.

```
import math

print(math.sqrt(144))
print(math.pi)
```

OUTPUT

```
12.0
3.141592653589793
```

- `import math` loads the module.
- `math.sqrt` means: the `sqrt` function that lives inside `math`. The **dot** goes inside the module.

Ways to import

STYLE	HOW YOU USE IT
`import math`	`math.sqrt(16)`
`from math import sqrt`	`sqrt(16)`
`from math import sqrt, pi`	`sqrt(16)` and `pi`
`import math as m`	`m.sqrt(16)`
`from math import *`	everything, with no `math.` (avoid this)

```
import math
print(math.sqrt(16))

from math import sqrt, pi
print(sqrt(25))
print(pi)

import math as m
print(m.floor(3.9))
```

OUTPUT

```
4.0
5.0
3.141592653589793
3
```

Avoid import *

`from math import *` pours every name into your program. You cannot tell where a name came from, and it can hide your own names. Import only what you need.

Put all your `import` lines at the **top** of the file.

The math module

NAME	WHAT IT GIVES	EXAMPLE
`math.sqrt(x)`	square root	`sqrt(49)` is `7.0`
`math.pi`	the number pi	`3.14159...`
`math.floor(x)`	round **down**	`floor(3.9)` is `3`
`math.ceil(x)`	round **up**	`ceil(3.1)` is `4`
`math.factorial(n)`	n times n-1 ... times 1	`factorial(5)` is `120`
`math.gcd(a, b)`	biggest number that divides both	`gcd(12, 18)` is `6`
`math.pow(a, b)`	a to the power b (a float)	`pow(2, 3)` is `8.0`

```
import math

print(math.sqrt(49))
print(math.floor(3.9), math.ceil(3.1))
print(math.factorial(5))
print(math.gcd(12, 18))
print(math.pow(2, 3))
```

OUTPUT

```
7.0
3 4
120
6
8.0
```

floor, ceil and round

```
import math

x = 2.5
print(math.floor(x))
print(math.ceil(x))
print(round(x))
print(round(3.5))
print(round(3.14159, 2))
```

OUTPUT

```
2
3
2
4
3.14
```

round() is a little surprising

`round(2.5)` gives `2` and `round(3.5)` gives `4`. Python rounds a half to the **nearest even** number. Use `floor` or `ceil` when you need a fixed direction.

Example: area of a circle

```
import math

radius = 7
area = math.pi * radius ** 2
print("Area:", round(area, 2))
```

OUTPUT

```
Area: 153.94
```

The random module

The `random` module makes **random** numbers and choices: games, quizzes, passwords, samples.

NAME	WHAT IT DOES
`random.random()`	a float from 0.0 up to (not including) 1.0
`random.randint(a, b)`	a whole number from a to b, **both included**
`random.uniform(a, b)`	a float between a and b
`random.choice(items)`	one random item
`random.sample(items, k)`	k different items
`random.shuffle(items)`	mixes a list **in place**
`random.seed(n)`	makes the "random" numbers repeat

The numbers change every run, so this box prints different things each time you press **Run**.

```
import random

print(random.randint(1, 6))
print(random.choice(["red", "green", "blue"]))
print(random.random())
```

```
OUTPUT
6
blue
0.7592327975413334
```

shuffle and sample

```
import random

cards = ["A", "K", "Q", "J", "10"]
random.shuffle(cards)
print(cards)

print(random.sample(range(1, 50), 6))
```

```
OUTPUT
['K', 'Q', 'J', 'A', '10']
[2, 10, 36, 15, 9, 24]
```

- `shuffle` **changes** the list and returns `None`. Do not write `cards = random.shuffle(cards)`.
- `sample` never repeats an item, so it is good for lottery numbers or picking winners.

seed: the same "random" numbers again

Computers do not make truly random numbers. They follow a recipe. `seed` picks the **starting point**, so the same seed gives the **same sequence**. This is useful for testing.

```
import random

random.seed(10)
first = [random.randint(1, 100) for _ in range(3)]

random.seed(10)
second = [random.randint(1, 100) for _ in range(3)]

print(first == second)
```

OUTPUT

```
True
```

Examples with random

Roll two dice

```
import random

d1 = random.randint(1, 6)
d2 = random.randint(1, 6)
print("You rolled", d1, "and", d2)
print("Total:", d1 + d2)
```

OUTPUT

```
You rolled 4 and 5
Total: 9
```

A one-time password (OTP)

```
import random

otp = ""
for _ in range(6):
    otp += str(random.randint(0, 9))
print("Your OTP is", otp)
```

OUTPUT

```
Your OTP is 037742
```

A random password

```
import random
import string

letters = string.ascii_letters + string.digits
password = "".join(random.choice(letters) for _ in range(10))
print(password)
```

OUTPUT

```
cHFuep88Vx
```

- `string.ascii_letters` is all the letters, and `string.digits` is `"0123456789"`.
- `"".join(...)` glues the 10 chosen characters into one string.

Heads or tails, 1000 times

```
import random

heads = 0
for _ in range(1000):
    if random.choice(["H", "T"]) == "H":
        heads += 1
print("Heads:", heads)
print("Tails:", 1000 - heads)
```

OUTPUT

```
Heads: 496
Tails: 504
```

Not for real security

`random` is fine for games and practice, but it is **not safe for real passwords or bank codes**. For those, Python has the `secrets` module.

The datetime module

The `datetime` module works with **dates and times**. It has three main parts:

NAME	WHAT IT HOLDS	EXAMPLE
`date`	year, month, day	`date(2026, 8, 15)`
`datetime`	date and time together	`datetime(2026, 8, 15, 9, 30)`
`timedelta`	a **length of time**	`timedelta(days=7)`

```
from datetime import date, datetime

d = date(2026, 8, 15)
print(d)
print(d.year, d.month, d.day)

dt = datetime(2026, 8, 15, 9, 30)
print(dt)
print(dt.hour, dt.minute)
```

OUTPUT

```
2026-08-15
2026 8 15
2026-08-15 09:30:00
9 30
```

Today and now

`date.today()` and `datetime.now()` read the **computer clock**, so the answer is different every time. This box only checks the type and the year.

```
from datetime import date, datetime

today = date.today()
now = datetime.now()
print(type(today))
print(today.year >= 2025)
print(type(now))
```

OUTPUT

```
<class 'datetime.date'>
True
<class 'datetime.datetime'>
```

Dates and timedelta

Subtract two dates to get a `timedelta`. Add a `timedelta` to a date to move forward or back.

```
from datetime import date, timedelta

start = date(2026, 1, 1)
end = date(2026, 3, 1)
gap = end - start
print(gap)
print(gap.days)

print(start + timedelta(days=30))
print(end - timedelta(weeks=1))
```

OUTPUT

```
59 days, 0:00:00
59
2026-01-31
2026-02-22
```

Weekday name

```
from datetime import date

d = date(2026, 8, 15)
print(d.weekday())
print(d.strftime("%A"))
```

OUTPUT

```
5
Saturday
```

`weekday()` gives a number: Monday is `0` and Sunday is `6`.

Formatting and reading dates

`strftime` turns a date **into text** in the shape you want. `strptime` does the reverse: it reads text **into a date**.

CODE	MEANING	EXAMPLE
`%d`	day (2 digits)	`05`
`%m`	month number	`08`
`%B`	month name	`August`
`%b`	short month	`Aug`
`%Y`	year (4 digits)	`2026`
`%A`	weekday name	`Saturday`
`%H:%M`	hour and minute (24 hour)	`09:30`

```
from datetime import datetime

dt = datetime(2026, 8, 5, 9, 30)
print(dt.strftime("%d/%m/%Y"))
print(dt.strftime("%d %B %Y"))
print(dt.strftime("%A, %H:%M"))
```

OUTPUT

```
05/08/2026
05 August 2026
Wednesday, 09:30
```

Reading a date from text

```
from datetime import datetime

text = "25/12/2026"
d = datetime.strptime(text, "%d/%m/%Y")
print(d)
print(d.strftime("%A"))
```

OUTPUT

```
2026-12-25 00:00:00
Friday
```

Work out the output first, then press Run to check

```
from datetime import date

born = date(2000, 6, 15)
check = date(2026, 6, 14)
age = check.year - born.year
if (check.month, check.day) < (born.month, born.day):
    age -= 1
print("Age:", age)
```

The `if` line lowers the age by one when the birthday has **not happened yet** this year. Tuples compare month first, then day.

More useful modules

MODULE	USE IT FOR
`math`	maths functions
`random`	random numbers and choices
`datetime`	dates and times
`statistics`	mean, median, mode
`collections`	`Counter` and other special containers
`time`	time in seconds, `sleep`
`os`	files, folders and the system
`sys`	Python itself (version, arguments)
`json`	reading and writing JSON

statistics

```
import statistics

marks = [70, 85, 85, 90, 100]
print(statistics.mean(marks))
print(statistics.median(marks))
print(statistics.mode(marks))
```

OUTPUT

```
86
85
85
```

collections.Counter

`Counter` does the counting from the dictionary session in one line.

```
from collections import Counter

word = "banana"
counts = Counter(word)
print(counts)
print(counts.most_common(1))
```

```
OUTPUT
Counter({'a': 3, 'n': 2, 'b': 1})
[('a', 3)]
```

time

```
import time

start = time.time()
total = sum(range(1_000_000))
took = time.time() - start
print(total)
print(took < 5)
```

```
OUTPUT
499999500000
True
```

`time.time()` is the number of seconds since 1 January 1970. Subtract two readings to time a piece of code. `time.sleep(2)` makes the program wait 2 seconds.

Finding what is inside a module

You do not need to remember everything. Python can tell you what a module has.

```
import math

print(math.sqrt.__doc__)
print("floor" in dir(math))
print(len(dir(math)) > 40)
```

```
OUTPUT
Return the square root of x.
True
True
```

- `dir(module)` lists the names inside a module.

- `help(module.name)` shows the full help text (best in a terminal or IDLE).
- The Python documentation at docs.python.org has an example for every module.

Writing your own module

Any `.py` file is a module. Save the code you reuse in one file, and import it from another file **in the same folder**.

File 1: shapes.py

```
PI = 3.14159

def circle_area(r):
    return PI * r ** 2

def square_area(side):
    return side * side
```

File 2: main.py (same folder)

```
import shapes

print(shapes.circle_area(2))
print(shapes.square_area(5))
```

OUTPUT

```
12.56636
25
```

This one needs two real files

The Try it boxes run one piece of code, so these two boxes are for reading. Try them on your own computer, or in your IDE, with both files in one folder.

__name__ == "__main__"

Every file has a built-in name called `__name__`. When you **run** the file directly it is `"__main__"`. When another file **imports** it, `__name__` is the module name instead.

```
print(__name__)

def main():
    print("Running as the main program")

if __name__ == "__main__":
    main()
```

OUTPUT

```
__main__
Running as the main program
```

Code under `if __name__ == "__main__":` runs **only when the file is started directly**, not when it is imported. That is how a module can have test code that stays quiet when imported.

Packages

A **package** is a **folder** of modules. Python knows a folder is a package when it holds a file called `__init__.py` (it can be empty).

A small package

```
my_project/
    main.py
    tools/
        __init__.py
        maths.py
        text.py
```

main.py

```
from tools import maths
from tools.text import shout

print(maths.double(4))
print(shout("hi"))
```

OUTPUT

```
8
HI!
```

- `tools.maths` means: the `maths` module inside the `tools` package.
- Big libraries such as `numpy` and `pandas` are packages with many modules inside.

pip install

The standard library is only the start. Thousands of free packages are shared on **PyPI**, the Python Package Index. **`pip`** is the tool that downloads and installs them.

pip runs in the terminal

Type `pip` commands in **Command Prompt** or **PowerShell** (the terminal), not inside a Python file and not in the Try it boxes on this site.

Terminal commands

```
pip --version
pip install requests
pip install requests==2.31.0
pip list
pip show requests
pip uninstall requests
```

COMMAND	WHAT IT DOES
`pip install name`	download and install a package
`pip install name==1.2.3`	install one exact version
`pip install --upgrade name`	update to the newest version
`pip list`	show everything installed
`pip show name`	details about one package
`pip uninstall name`	remove a package
`pip freeze`	list packages with exact versions

If "pip" is not found on Windows

Try `py -m pip install requests`. It runs pip through the Python launcher, so it works even when `pip` is not on the PATH.

After installing

Import an installed package just like a standard module.

Needs the internet and: pip install requests

```
import requests

reply = requests.get("https://www.python.org")
print(reply.status_code)
```

OUTPUT

```
200
```

`200` means "OK". That is the web server saying the page was found.

Virtual environments and requirements.txt

Different projects may need different versions of the same package. A **virtual environment** gives each project its **own private set of packages**.

Terminal commands (Windows)

```
python -m venv venv
venv\Scripts\activate
pip install requests
pip freeze > requirements.txt
deactivate
```

- `python -m venv venv` makes a folder called `venv` for this project.
- `activate` switches it on. The prompt starts with `(venv)`.
- `pip freeze > requirements.txt` saves the list of packages and versions.
- A friend can then run `pip install -r requirements.txt` to get the same packages.
- `deactivate` switches it off.

On Mac or Linux

Use `python3` instead of `python`, and activate with `source venv/bin/activate`.

Common mistakes

- **Forgetting to import.** Using `sqrt(9)` without `from math import sqrt` gives a `NameError`.
- **A misspelled or missing module** gives `ModuleNotFoundError`. For a package, you may need `pip install` first.
- **Naming your own file like a module.** A file called `random.py` or `math.py` hides the real one, and your imports break.
- **Writing `import math` but then calling `sqrt(9)`.** You need `math.sqrt(9)`.
- **Installing with a different Python than you run.** Use `py -m pip install` so they match.

Try it: this fails on purpose

```
import mathh

print(mathh.sqrt(4))
```

ERROR

```
  File "your code", line 1, in <module>
ModuleNotFoundError: No module named 'mathh'
```

Try it: this also fails on purpose

```
import math

print(sqrt(9))
```

OUTPUT

```
3.0
```

Practice

Your turn: print the area of a circle with radius 10, rounded to 1 decimal place

```
import math


```

Your turn: pick 3 different names at random from the list

```
import random

names = ["Aman", "Ankit", "Neha", "Rohit", "Sara"]
```

Your turn: print how many days from 1 Jan 2026 to 25 Dec 2026

```
from datetime import date


```

Homework and wrap-up

- Write a program that rolls **two dice 100 times** and counts how many times the total is 7.
- Write a function that makes a **random password** of any length.
- Ask for a birth date as text (`dd/mm/yyyy`), read it with `strptime`, and print the **age** and the **weekday** of birth.
- Make your own module `mytools.py` with two functions, then import it in another file.
- Install a package with `pip install` and print its version with `pip show`.

- Revise: **import styles**, `math`, `random`, `datetime`, **modules vs packages**, `pip`.

Next class

File handling: read, write, append, `with`, and working with text and CSV files.